

UPSTREAM ENERGY

Kingdom 2020 SDK® Developer Guide

April 2020



Kingdom 2020 SDK ® Developer Guide

April 28, 2020

© 1997-2020, IHS Markit and its affiliated and subsidiary companies, all rights reserved.

Kingdom® software and all of its components, AVOPAK®, CGMPAK®, LoadPAK®, PAKnotes®, IHS Markit Petra®, GeoSyn® Tunnel L+®, Tunnel O®, VelPAK®, VuPAK®, Kingdom 1D Forward Modeling®, Kingdom Colored Inversion™, The Kingdom Company®, Kingdom Data Management™, Kingdom DM Catalog Builder™, Kingdom Illuminator™, Kingdom Seeker™, are trademarks or registered trademarks of IHS Markit Inc.

This product, including software, data and documentation are licensed to the user for its internal business purposes only and may not be disclosed, disseminated, sold, licensed, copied, reproduced, translated or transferred to any third party.

IHS Markit Global Headquarters
4th floor, Ropemaker Place
25 Ropemaker Street
London
EC2Y 9LY
United Kingdom
Phone: +44 20 7260 2000

TRADEMARKS

IHS Markit and the IHS Markit logo are trademarks of IHS Markit. Other trademarks appearing in this publication are the property of IHS Markit or their respective owners.

Table of Contents

Table of Contents	ii
Overview.....	1
Kingdom Architecture	1
Access to Kingdom Data	1
How to Build Sample Projects	2
Pre-requisites	2
Kingdom Data Transfer / KDT (Sample 1)	2
KDT Project Location	2
Assemblies	2
Test Kingdom Projects	2
KDT Project Features	2
Step by Step Execution	4
SDK Best Practices (Sample 2)	122
Developing Your First Application using the Kingdom SDK	133
Step 1 – Licensing.....	133
License Intialization	133
License Activation	13
Step 2 – Set Up Project	188
Step 3 - Code	19
Set up Project Class	19
Access Data.....	19
Oracle.....	20
Deploying Your Application	200
Understanding Kingdom Data	211
Data Model.....	211
Data Model Categories Structure	288
Data Access	29
Read Data.....	30
By One ID	300
By Collection of IDs	300
Visible-only Flag.....	300
Projection.....	311
Projection & Visible-only Flag.....	311

Predicate.....	322
Well Subsets.....	322
Author Types	322
Author Ids.....	333
Navigation Properties.....	344
Create Data.....	355
Update Data	366
Create or Update Data	377
Delete Data	377
Use a Delete Parameter	377
Call DeleteObject() Method.....	388
Data Access Settings	388
Miscellaneous Methods.....	399
Formation Tops Author Priority	3939
Get Formation Tops in a Strat Column	3939
Get Formation Tops aliasing	3939
API Changes	400
Programming Guidelines and Best Practices	422
Appendix	444
Appendix A – Exceptions.....	444
SQL Server Express Exceptions (SseException)	444
Additional Exceptions	455

Overview

Kingdom SDK is a software development kit that provides data access between third-party applications and Kingdom project data. Third-party applications can access data in a simple-to-use .NET 4.6.1+ programming interface that abstracts out how Kingdom project data are physically stored. Kingdom SDK consists of assemblies and utilities based on .NET Framework 4.6.1+, with additional external dependencies that are included in the package as well.

Kingdom SDK can be used to support several user scenarios; for example, a third-party application can be extended to pull log data programmatically from Kingdom, perform some application-specific operations, and then save the data back to Kingdom. All of this can be done without opening the Kingdom application.

Another example of SDK usage is to create a custom data aggregation utility to read Kingdom data programmatically. These data can then be used for automating additional workflows such as creating custom reports, logics, integrated mapping, and visualizations.

Kingdom Architecture

Kingdom data architecture consists of three main layers: Application, Data Access, and Data Storage. The application layer consists of the Kingdom application suite, as well as applications or tools that our partners and customers develop. These applications access physical data in Kingdom from the database, or from a file via the data access layer, which is provided by the Kingdom SDK and other proprietary technologies. Data validation is also done in the data access layer.

Access to Kingdom Data

The following Kingdom project data types can be accessed through Kingdom SDK:

- Well Header
- Well Location
- Deviation Survey
- Well Logs
- Formation Tops
- Production Data
- Time-Depth Charts
- Zones
- Grids
- Contours
- Cultures
- Completions
- Production Tests
- Formation Tests
- Well Tests
- Perforations
- Control Point Sets
- Synthetics
- Cores
- Intervals
- Faults
- Fault Cuts
- Cumulative Production
- Drilling Events
- Seismic Navigation Data
- Horizons

* Read access is available on most entities. Write access is limited only to database entities and certain file-based entities. For a table with all supported entities, see [Understanding Kingdom Data](#).

How to Build Sample Projects

Kingdom SDK comes with two sample projects to help you jump start your development:

- KingdomDataTransfer
- SDKBestPractices

Pre-requisites

- Visual Studio 2012+

Kingdom Data Transfer / KDT (Sample 1)

The first sample project, KingdomDataTransfer, is a .NET-based Windows application that illustrates the use of Kingdom SDK to read and write Kingdom project data. It allows you to transfer several objects with a set of properties. If you need to access additional properties of these Kingdom data objects, please refer to the Kingdom SDK API & Object Model documentation or consult with the Kingdom SDK support team.

KDT Project Location

\\Sample Projects\\KingdomDataTransfer\\KingdomDataTransfer.sln

Assemblies

All SDK assemblies are available in \\Assemblies and should already be referenced from the project.

Copy below assemblies to the \\Sample Projects\\KingdomDataTransfer\\KingdomDataTransfer\\Bin\\Debug folder

- | | |
|---------------------------------|--|
| • OpenKINGDOMUtils.dll | • Oracle.ManagedDataAccess.EntityFramework.dll |
| • SeismicDevKit.dll | • System.Data.SqlServerCe.dll |
| • Smt.Interop.Brick.dll | • IHS.Kingdom.SDK.EntityDbConfig.dll |
| • Smt.Interop.Segy.dll | • LicenseClient.dll |
| • EntityFramework.dll | • LicenseClientImplx64.dll |
| • EntityFramework.SqlServer.dll | • LicenseClientImplx86.dll |
| • Oracle.ManagedDataAccess.dll | • LicenseClientResources.dll |

Test Kingdom Projects

Make sure that both Kingdom projects (source and destination) have been upgraded to Kingdom 2020. To upgrade, open the project with Kingdom 2020.

KDT Project Features

- From source project
 - Read and display boreholes Id, UWI, Well Name, Borehole Name, Well Number
 - Read and display Digital Log names and Ids
 - Read and display Formation Top names and Ids
 - Read and display Picked Interval names and Ids
 - Read and display Interval names and Ids
 - Read and display Fault Cuts names and Ids
 - Read and display Well Subset name, type
 - Read and display Survey name and Survey Type (2D or 3D).
 - When clicked on the survey, read and display seismic data's ZType, Seismic Type Name and Survey Name

- Read and display Horizon names and Ids
- Read and display Grid names and Ids
- Read and display Fault names and Ids
- Get Ids of all selected data

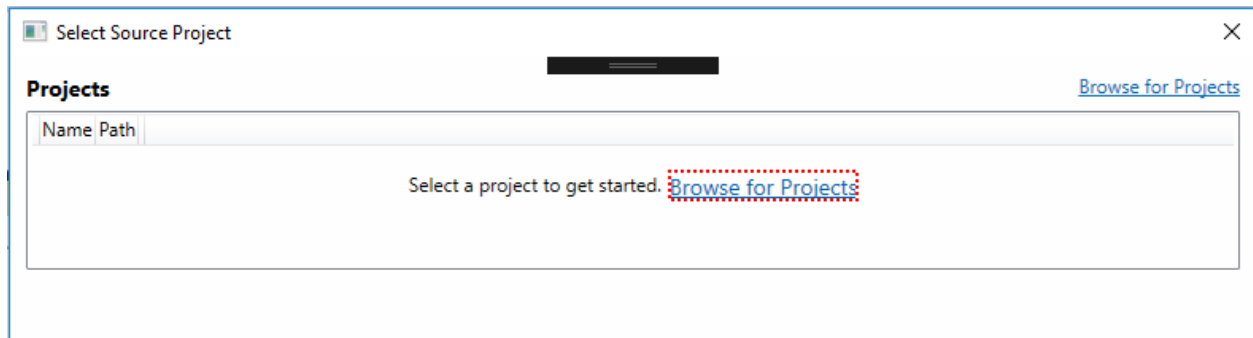
For each category, get details of the selected data through below mentioned SDK object and populate it to KingdomDataTransfer (KDT) object; and map them back to SDK objects and save to destination project.

KingdomDataTransfer Entities	Kingdom SDK Entities
Borehole	Well, Borehole, ProducingField
Well Subset	Well, Borehole, ProducingField
Deviation Survey	DeviationSurvey, ActiveDeviation
Digital Log	LogData, LogType
Formation Tops	FormationTopPick, FormationTopName
Picked Interval	PickedInterval, PickedIntervalName
Interval	IntervalRecord, IntervalName, IntervalAttributes, IntervalTextValue, IntervalNumericValue, IntervalDateTimeValue
Fault Cuts	FaultCutPick, FaultCutName
Faults	Fault, NewFaultSegment, Faultvertex
Horizons	Horizon, SeismicType
Grids	Grids
Surveys	SeismicLine2D, SeismicSurvey3D
Seismic	SeismicData2D, SeismicData3D

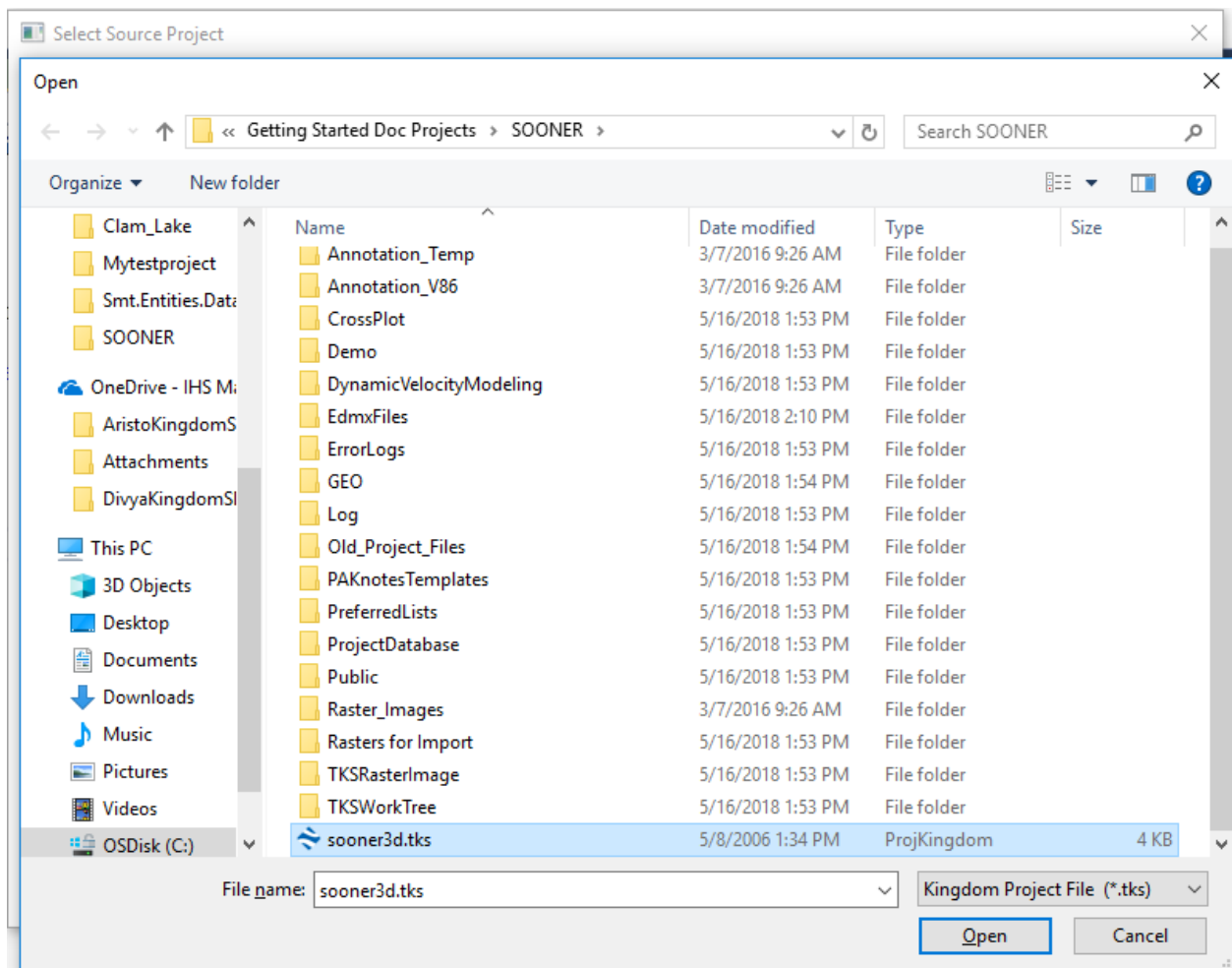
- KDT can read and write Borehole with Deviation surveys, Digital logs, Formation tops, Picked intervals, Intervals and Fault cuts
- KDT can read and write Faults with Surveys
- KDT can read and write Horizons with Surveys, Fault Polygons, and Faults associated with the Fault Polygons
- KDT can read and write Grids
- KDT can read and write Surveys with Seismic data.
- KDT can write all boreholes in the well subset but cannot write well subsets themselves

Step by Step Execution

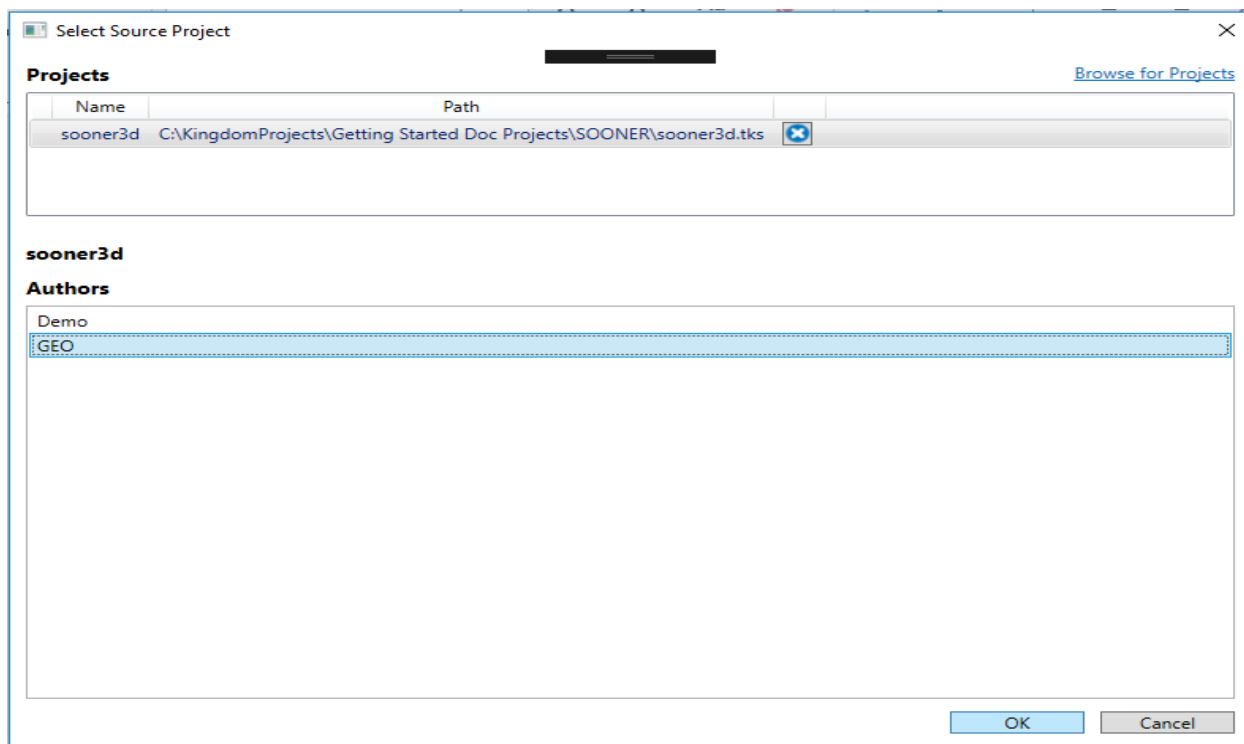
1. In Visual Studio 2012+, run the solution. The **Select Source Project** window opens. Click **Browse for Projects**.



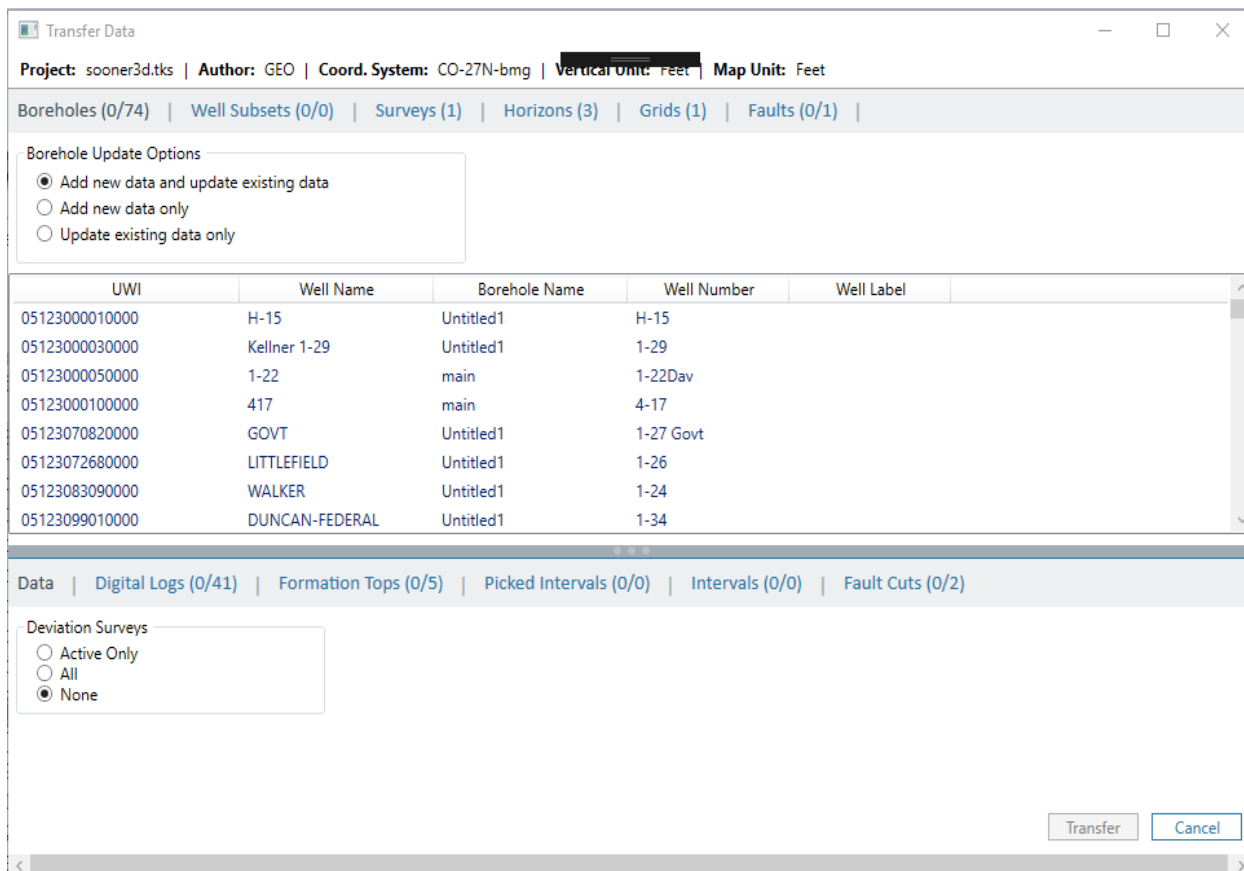
2. Select source Kingdom project (*.tks) and click **Open** (The project must be compatible with Kingdom 2020).



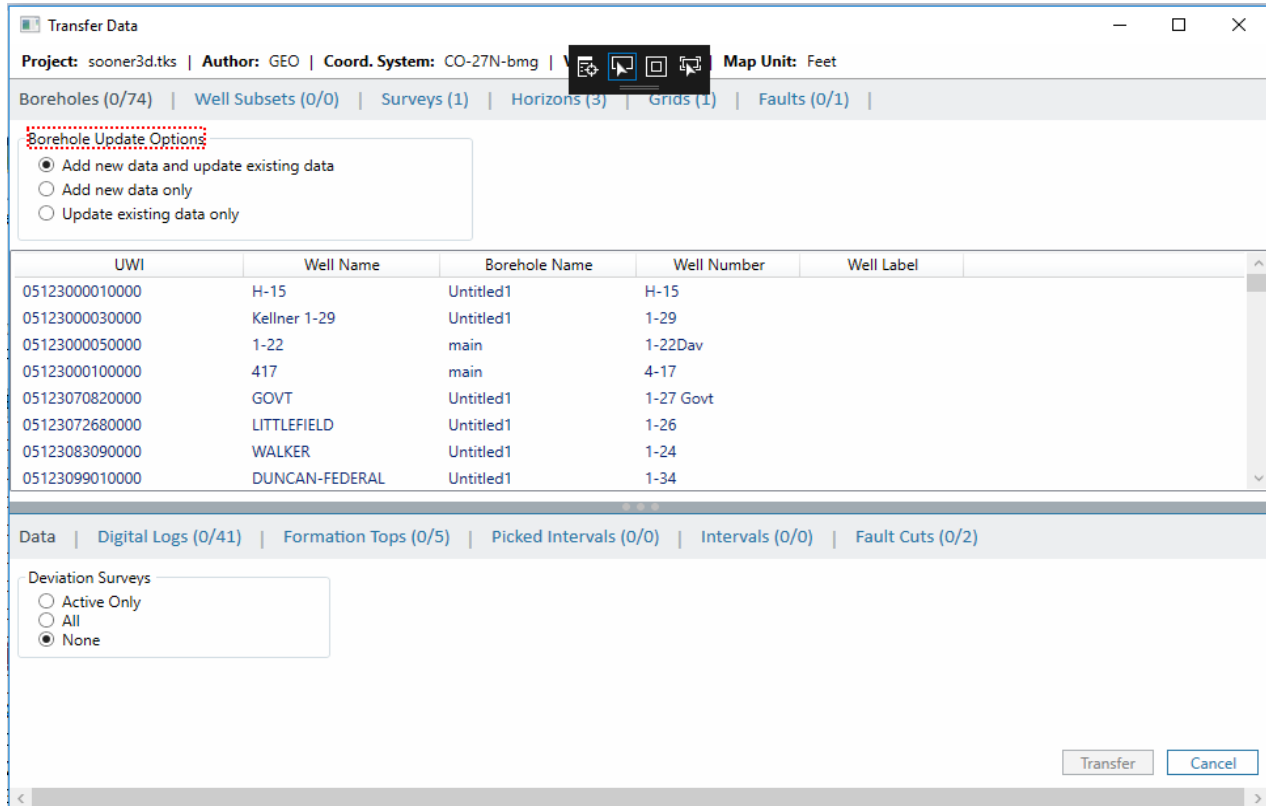
3. Select the author in the source project from which you want to transfer the authored data and click **OK**.



The **Transfer Data** dialog box opens.



- On the **Boreholes** tab select the first option, **Add new data and update existing data**.



Transfer Data

Project: sooner3d.tks | Author: GEO | Coord. System: CO-27N-bmg | Map Unit: Feet

Boreholes (0/74) | Well Subsets (0/0) | Surveys (1) | Horizons (3) | Grids (1) | Faults (0/1)

Borehole Update Options

- ☒ Add new data and update existing data
- ☐ Add new data only
- ☐ Update existing data only

UWI	Well Name	Borehole Name	Well Number	Well Label
05123000010000	H-15	Untitled1	H-15	
05123000030000	Kellner 1-29	Untitled1	1-29	
05123000050000	1-22	main	1-22Dav	
05123000100000	417	main	4-17	
05123070820000	GOVT	Untitled1	1-27 Govt	
05123072680000	LITTLEFIELD	Untitled1	1-26	
05123083090000	WALKER	Untitled1	1-24	
05123099010000	DUNCAN-FEDERAL	Untitled1	1-34	

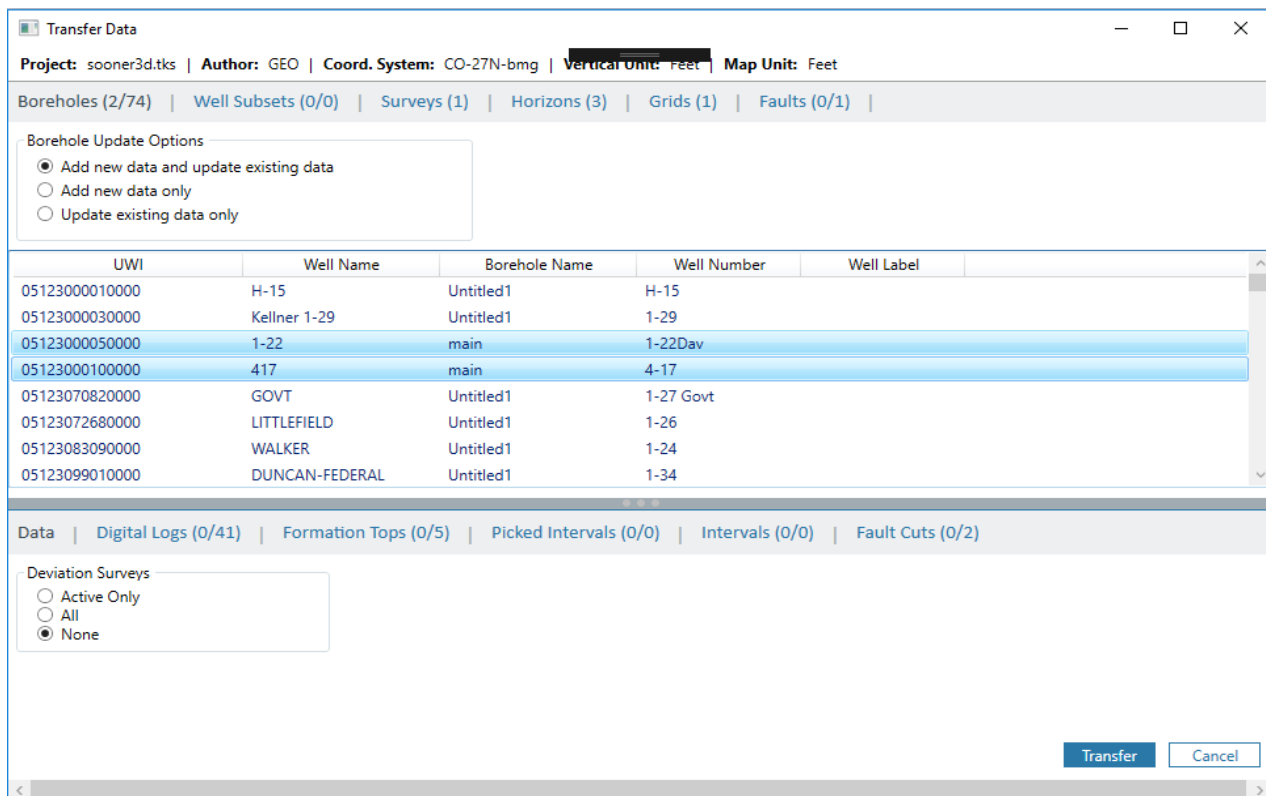
Data | Digital Logs (0/41) | Formation Tops (0/5) | Picked Intervals (0/0) | Intervals (0/0) | Fault Cuts (0/2)

Deviation Surveys

- ☐ Active Only
- ☐ All
- ☒ None

Transfer Cancel

- Select one or more boreholes to enable data transfer.



Transfer Data

Project: sooner3d.tks | Author: GEO | Coord. System: CO-27N-bmg | Map Unit: Feet

Boreholes (2/74) | Well Subsets (0/0) | Surveys (1) | Horizons (3) | Grids (1) | Faults (0/1)

Borehole Update Options

- ☒ Add new data and update existing data
- ☐ Add new data only
- ☐ Update existing data only

UWI	Well Name	Borehole Name	Well Number	Well Label
05123000010000	H-15	Untitled1	H-15	
05123000030000	Kellner 1-29	Untitled1	1-29	
05123000050000	1-22	main	1-22Dav	
05123000100000	417	main	4-17	
05123070820000	GOVT	Untitled1	1-27 Govt	
05123072680000	LITTLEFIELD	Untitled1	1-26	
05123083090000	WALKER	Untitled1	1-24	
05123099010000	DUNCAN-FEDERAL	Untitled1	1-34	

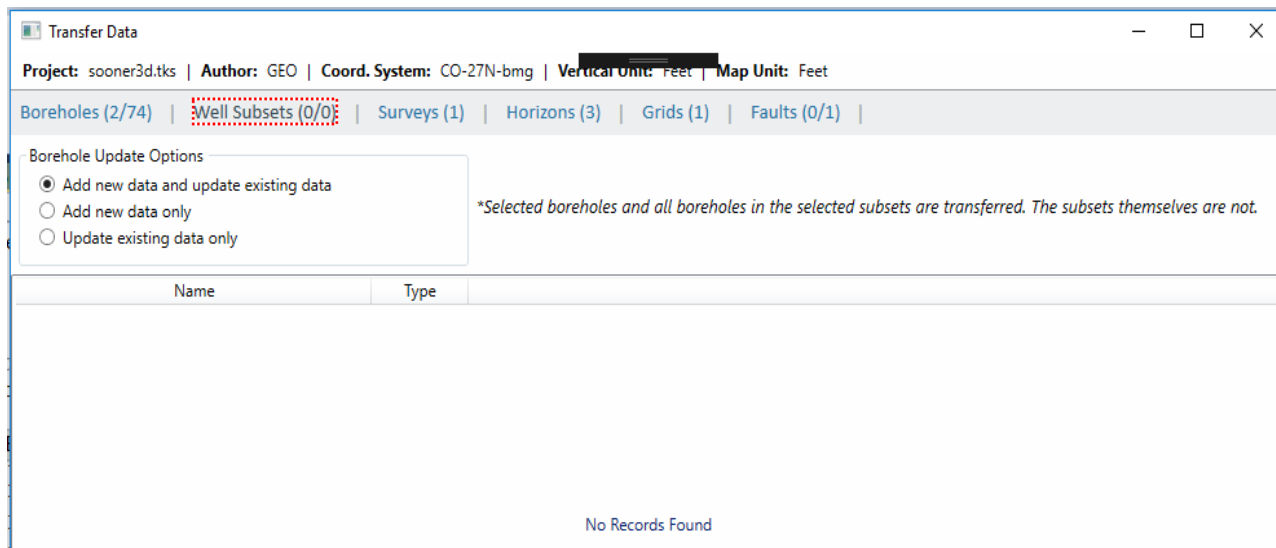
Data | Digital Logs (0/41) | Formation Tops (0/5) | Picked Intervals (0/0) | Intervals (0/0) | Fault Cuts (0/2)

Deviation Surveys

- ☐ Active Only
- ☐ All
- ☒ None

Transfer Cancel

6. Select the **Well Subsets** tab and select the subsets that you want to transfer.



Transfer Data

Project: sooner3d.tks | Author: GEO | Coord. System: CO-27N-bmg | Vertical Unit: Feet | Map Unit: Feet

Boreholes (2/74) | **Well Subsets (0/0)** | Surveys (1) | Horizons (3) | Grids (1) | Faults (0/1)

Borehole Update Options

☒ Add new data and update existing data
☐ Add new data only
☐ Update existing data only

**Selected boreholes and all boreholes in the selected subsets are transferred. The subsets themselves are not.*

Name	Type
No Records Found	

7. Click on the **Data** tab below the list of subsets and select a **Deviation Surveys** option:

- Active Only—only copy active deviation surveys
- All—copy all deviation surveys, active and inactive
- None—do not copy any deviation surveys. This is the default.



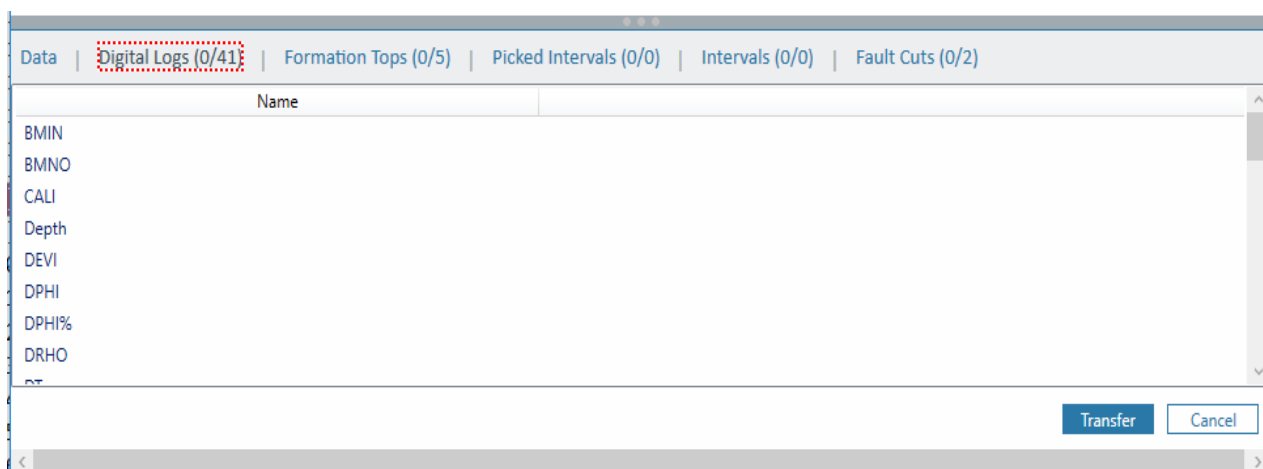
Data | Digital Logs (0/41) | Formation Tops (0/5) | Picked Intervals (0/0) | Intervals (0/0) | Fault Cuts (0/2)

Deviation Surveys

☐ Active Only
☐ All
☒ None

Transfer Cancel

8. Beside the Data tab, click on the **Digital Logs** tab and select the logs you want to transfer.



Data | **Digital Logs (0/41)** | Formation Tops (0/5) | Picked Intervals (0/0) | Intervals (0/0) | Fault Cuts (0/2)

Name
BMIN
BMNO
CALI
Depth
DEVI
DPHI
DPHI%
DRHO
DT

Transfer Cancel

9. Do the same for the remaining bottom tables:

- a. Formation Tops
- b. Picked Intervals
- c. Intervals Fault Cuts

Navigation: Data | Digital Logs (1/41) | **Formation Tops (0/5)** | Picked Intervals (0/0) | Intervals (0/0) | Fault Cuts (0/2)

Name
Base D sand
D Sand
J Sand
J Silt
X_MKR

Transfer Cancel

Navigation: Data | Digital Logs (1/41) | Formation Tops (0/5) | **Picked Intervals (0/0)** | Intervals (0/0) | Fault Cuts (0/2)

Name	
No Records Found	

Transfer Cancel

Navigation: Data | Digital Logs (1/41) | Formation Tops (0/5) | Picked Intervals (0/0) | **Intervals (0/0)** | Fault Cuts (0/2)

Interval Update Options

Interval Data cannot be updated.
It creates a new record everytime when it is transferred.
Possibility of duplicate entry*

Name	
No Records Found	

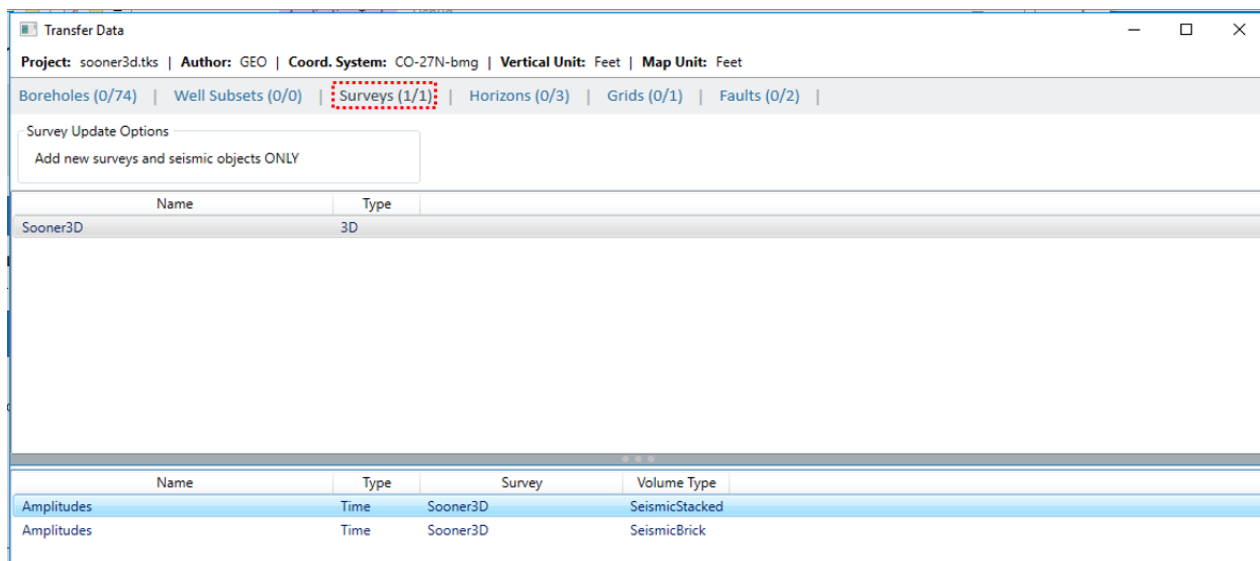
Transfer Cancel

Navigation: Data | Digital Logs (1/41) | Formation Tops (0/5) | Picked Intervals (0/0) | Intervals (0/0) | **Fault Cuts (0/2)**

Name
Test
Test2

Transfer Cancel

- Click on the **Surveys** tab at the top to view available surveys. By selecting the survey, view the seismic data types (in the bottom pane) of the survey. Select one or more surveys and seismic data types to transfer.



Project: sooner3d.tks | Author: GEO | Coord. System: CO-27N-bmg | Vertical Unit: Feet | Map Unit: Feet

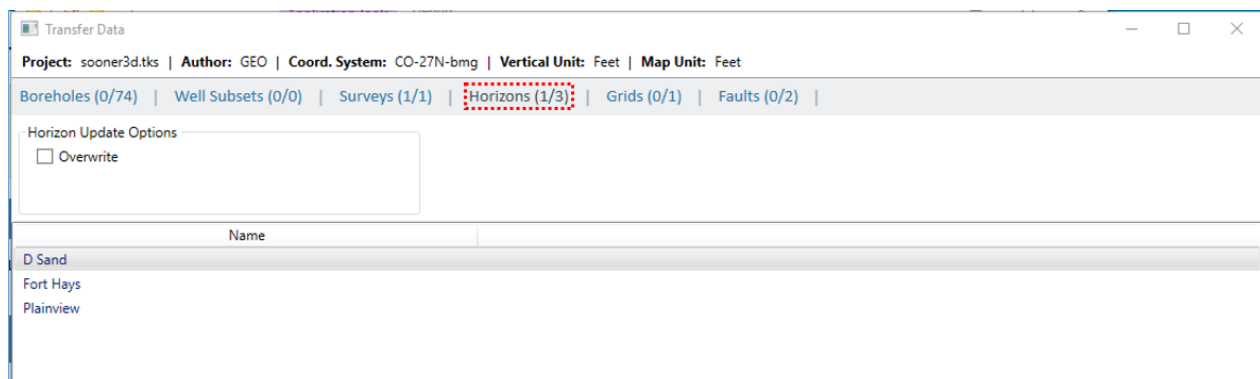
Boreholes (0/74) | Well Subsets (0/0) | **Surveys (1/1)** | Horizons (0/3) | Grids (0/1) | Faults (0/2)

Survey Update Options
☐ Add new surveys and seismic objects ONLY

Name	Type
Sooner3D	3D

Name	Type	Survey	Volume Type
Amplitudes	Time	Sooner3D	SeismicStacked
Amplitudes	Time	Sooner3D	SeismicBrick

- Click on the **Horizon**, **Fault** and **Grid** tab and view the names of available Horizons, Faults and Grids respectively. Select one or more names to transfer.

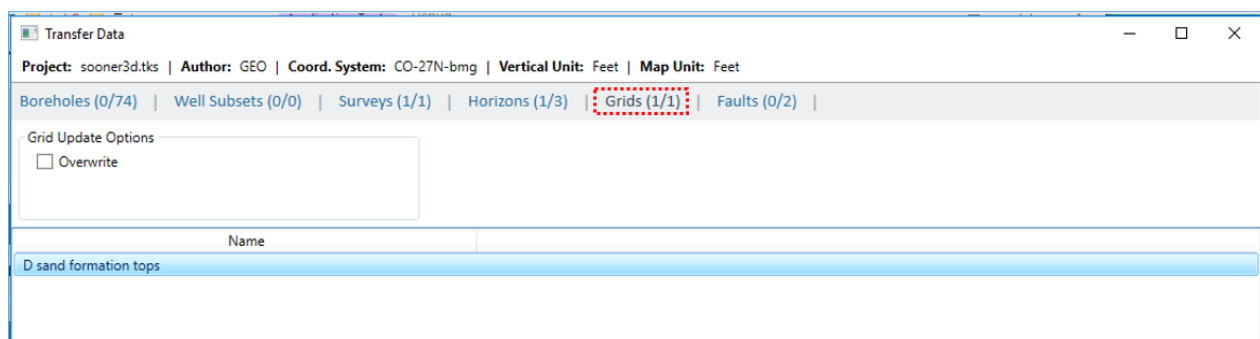


Project: sooner3d.tks | Author: GEO | Coord. System: CO-27N-bmg | Vertical Unit: Feet | Map Unit: Feet

Boreholes (0/74) | Well Subsets (0/0) | Surveys (1/1) | **Horizons (1/3)** | Grids (0/1) | Faults (0/2)

Horizon Update Options
☐ Overwrite

Name
D Sand
Fort Hays
Plainview

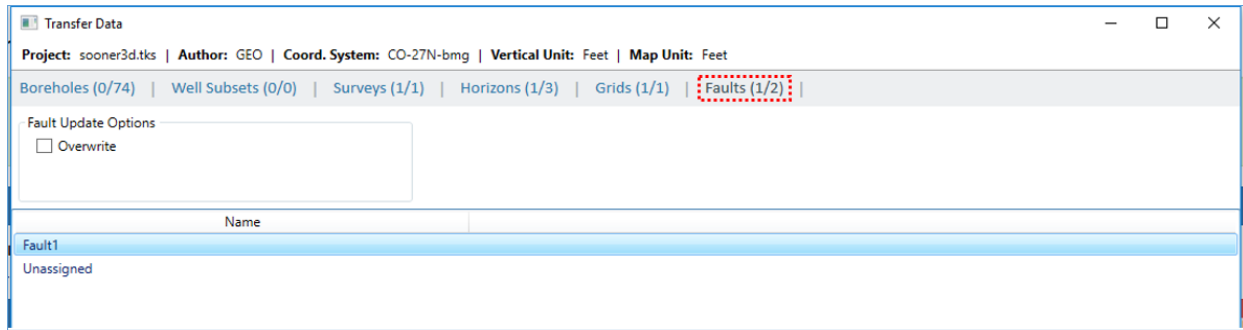


Project: sooner3d.tks | Author: GEO | Coord. System: CO-27N-bmg | Vertical Unit: Feet | Map Unit: Feet

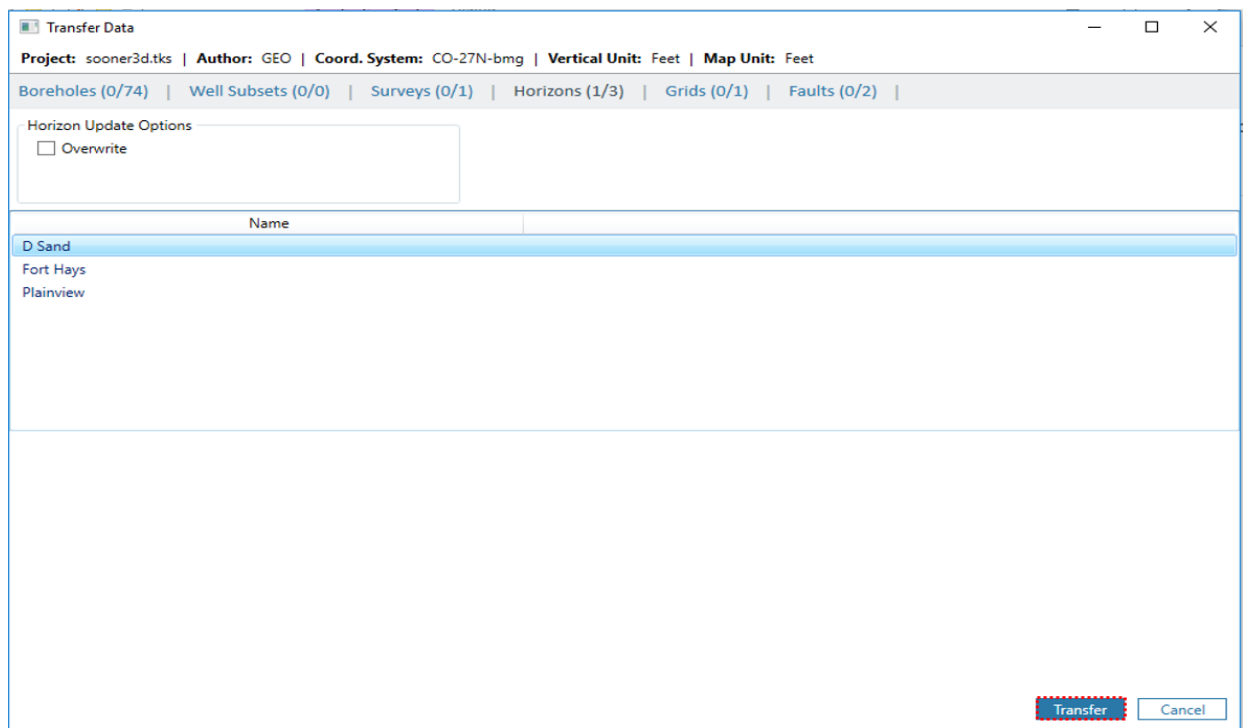
Boreholes (0/74) | Well Subsets (0/0) | Surveys (1/1) | Horizons (1/3) | **Grids (1/1)** | Faults (0/2)

Grid Update Options
☐ Overwrite

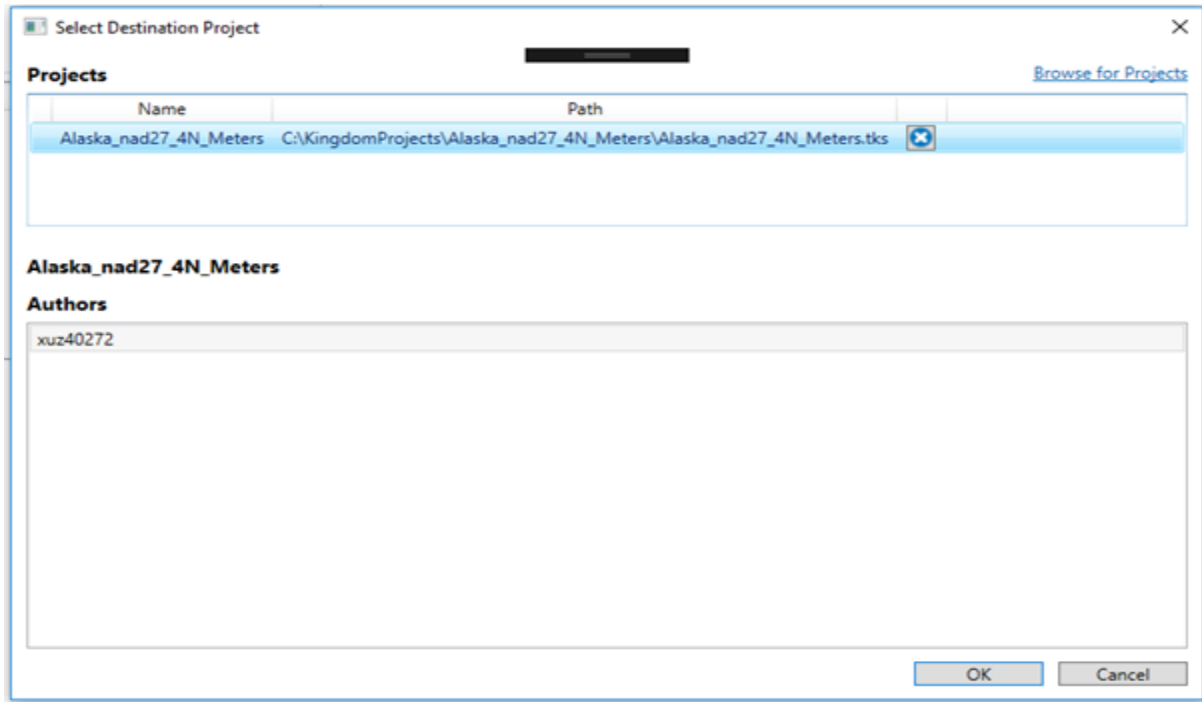
Name
D sand formation tops



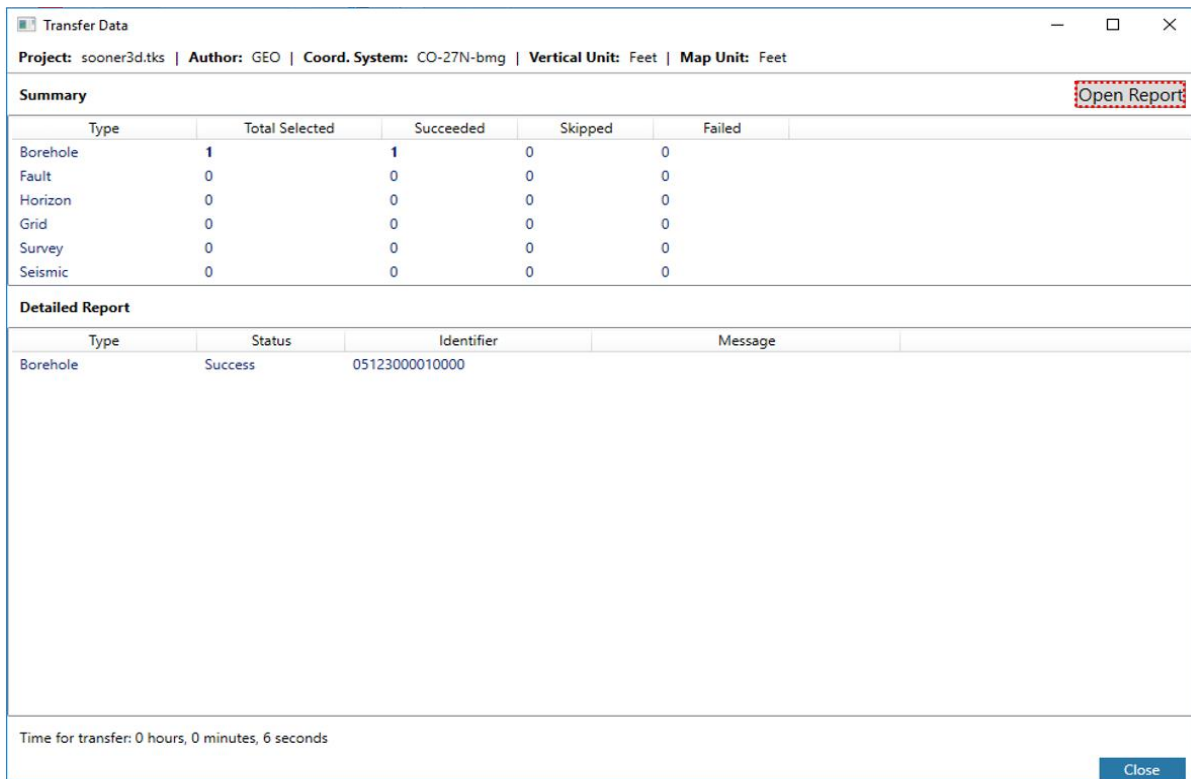
12. After selecting the data to transfer, click **Transfer**.



13. In the **Select Destination Project** dialog box, click **Browse for Projects** and navigate to the destination project. (Source and destination project cannot be the same) Select the author in the destination project that you want to transfer the interpretation data to (Boreholes, Well Subsets, intervals, and picked intervals) and click **OK**.



14. In the **Transfer Data** summary, click **Open Report** to view the details of the data transfer in Excel.



SDK Best Practices (Sample 2)

The SDKBestPractices project is a unit test project that also shows you how to work with the Kingdom SDK. It contains several best practice examples including connecting to a Kingdom project, reading simple / complex objects, and writing back to Kingdom data.

Follow these steps to load and build the project:

1. Run `lsinit.exe` as Administrator.
2. Open SDKBestPractices solution/project with Visual Studio 2012+.
3. Configure parameters in Helper.cs class.

Parameters	Description
Project Path	The full path to the Kingdom project file that you want to access, i.e. C:\DemoProjects\SOONER\sooner3d.tks
Author name	The name of the Author to be used for logging in as defined in the corresponding project, i.e. Demo (case-sensitive)
DB username	The name of the database user. Needed if the project is not using Auto SSE (SQL Server Express) database.
Password	The password of the database user. Needed if the project is not using Auto SSE (SQL Server Express) database.

Note: these parameters are literal strings enclosed by square brackets.

4. [Set up the license.](#)
5. Build the project and run unit tests.

Developing Your First Application using the Kingdom SDK

1. Licensing	Initialize SDK License: Run /Utilities/lsinit.exe as Administrator. If no error message is thrown, then it has been successfully run. Configure and Activate SDK License Run /Utilities/License Activation/IHS.Kingdom.Licensing.Activate.exe and then follow the instruction on how to activate a network, stand-alone or commuter license
2. Setup	Reference .NET SDK libraries from your .NET project and copy other supporting libraries to the output folder.
3. Code	• Connect to the Kingdom Project • Create Kingdom Data Context • Perform read from/write to Kingdom data objects

Step 1 – Licensing

Kingdom SDK supports standalone, network, and commuter license schemes. To open a Kingdom project using the SDK, you must be licensed to run the SDK Runtime. The SDK first tries checking out a standalone or commuter license. If neither is found, it tries to check out a license from the network license server.

If you or the end-user of the SDK does not have a Kingdom license, then you need to run the license initialization and activation utilities. See below.

License Initialization

When developing applications using the Kingdom SDK, or if distributing the Kingdom SDK with your application, you need to initialize the license by running \Utilities\lsinit.exe as Administrator, which registers the required licensing information for that machine. You only need to initialize the license once on any single machine.

Note: If Kingdom installation is required to run your third-party application and is already installed, then this step can be skipped. The license initialization is already done by the Kingdom installer.

Third-party developers need to run this executable before using the SDK for the first time as well.

For end-user distribution of your applications, the execution of lsinit.exe should be added as a step in that application installer.

License Activation

After your license is initialized, the next step is to activate the license.

Note: If you have Kingdom installed and have a valid base license, you can skip this step, unless the SDK Runtime licenses are on a separate license server.

If you are using only the SDK Runtime license without a Kingdom license, you need to use the Kingdom SDK License Manager to activate the license. The License Manager comes with the Kingdom SDK package: \Utilities\License Activation\IHS.Kingdom.Licensing.Activate.exe.

Activating a Network License

1. In the Kingdom SDK License Manager, enter the IP address or DNS name of your Kingdom License Server and click **Connect & Save**.

The license manager checks to confirm that you have the license to run SDK Runtime. It also saves the license server information in your system. If this field is prepopulated, a network license server has already been set up. If the SDK Runtime licenses are on a different server, you will need to get that information from the license administrator.

2. When you have successfully connected, confirm the updated license information under the Status section. The information includes the license server name, type, and expiration date.

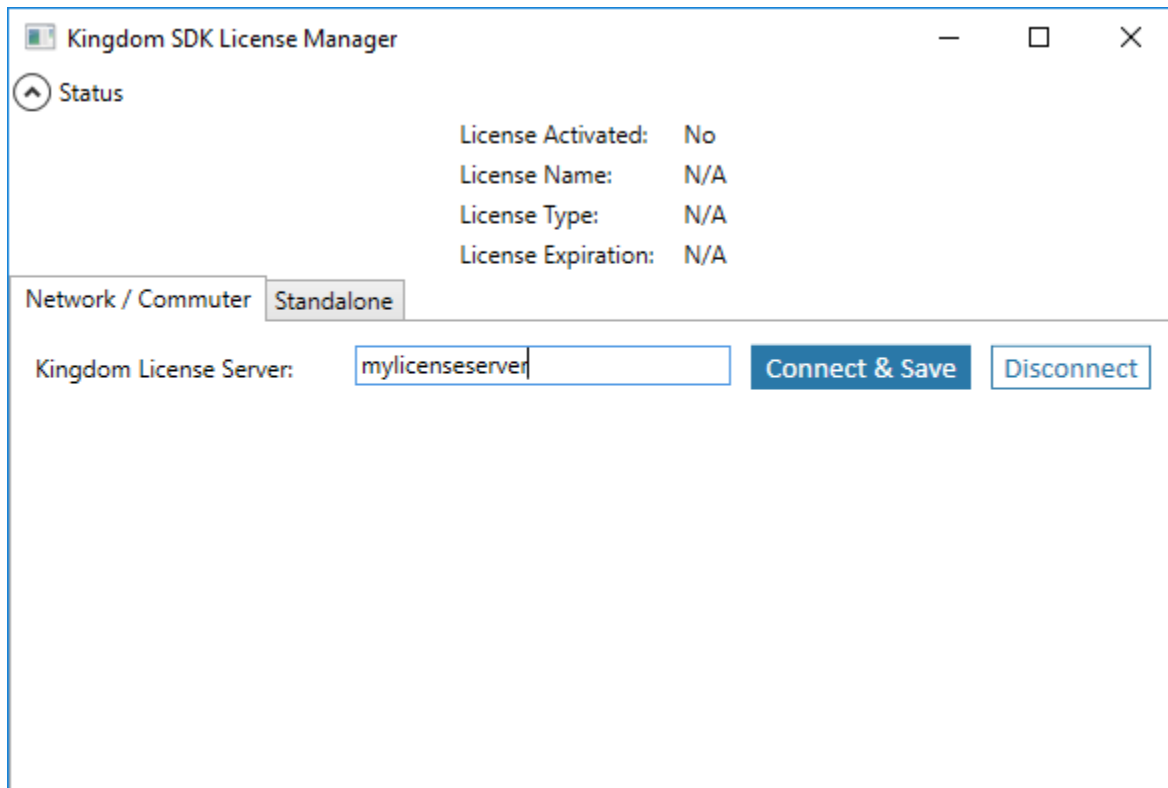


Figure 4-1: License Manager

Activating a Standalone License using Online Activation

1. If you are currently connected to a license server, click Disconnect to enable standalone license activation.
2. In the Kingdom SDK License Manager, click Online Activation.

Note: If you don't have a live Internet connection or have an issue with the online activation process, see [Manual Activation](#) for details, or follow the instructions in the License Manager dialog box.

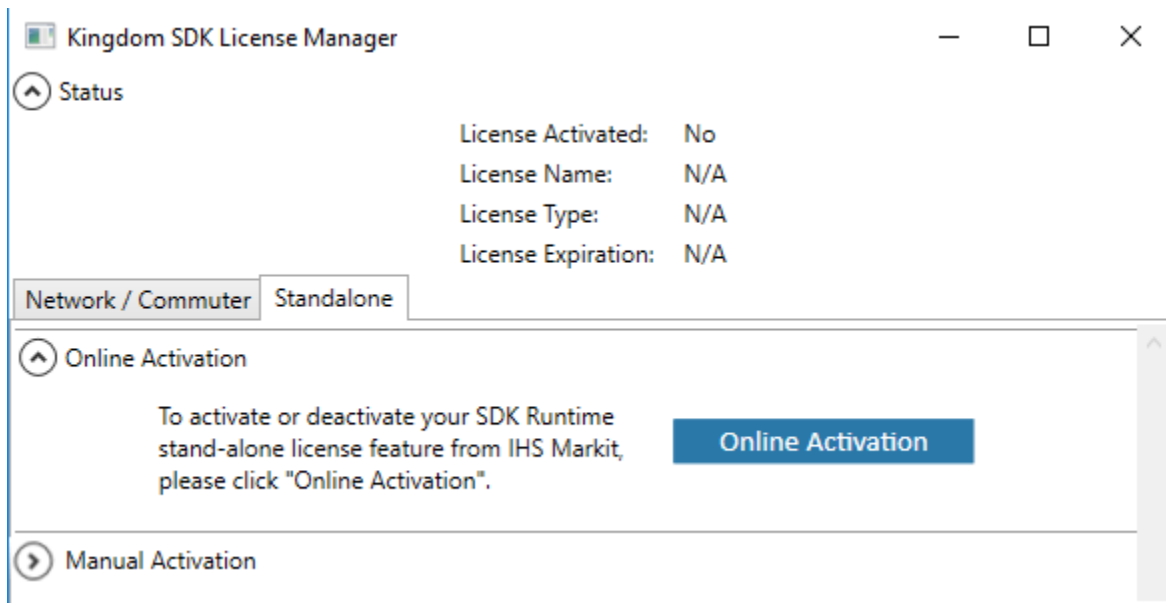


Figure 4-2: Standalone license

3. The SDK **Online Activation** dialog box is the same as the Kingdom Online Activation dialog box. Enter the **Entitlement Id** (EID) that was provided in the Entitlement Certificate email when you purchased Kingdom / SDK.

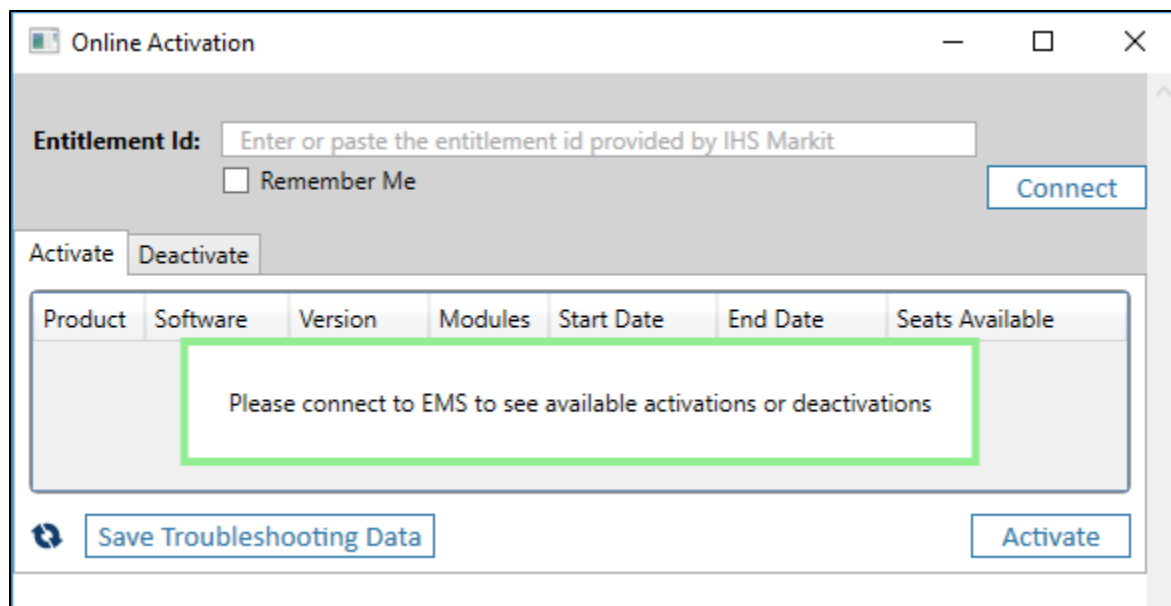


Figure 4-3: Stand-alone online activation

- After you enter the EID, all products included in the entitlement are listed. Select **SDK Runtime** and click **Activate**. If you have successfully activated the standalone license, the license information will be shown under the **Status** section with the proper license name, type and expiration date.

Activating a Standalone License Manually

If you do not have internet access, you have the following options:

Connect to the EMS on a different computer	On a computer that has internet access, connect to the Entitlement Management System at https://softwarelicensing.ihsmarkit.com/ems . You will need both your EID, and the computer code from the other computer. See Retrieving a license file from the Sentinel EMS for detailed instructions.
Email IHS Markit	Send an email to kingdomlicensing@ihsmarkit.com with your computer code in the subject field. IHS Markit will send you a license file (lserverc.txt) with the license string. Paste the string into the field in the Manual Activation section and click Activate.

➤ Manual Activation

Alternatively, you can connect to the Entitlement Management System at the following address: <https://softwarelicensing.ihsmarkit.com/ems>

or send IHS Markit an email with the Computer Code: kingdomlicensing@ihsmarkit.com then paste the license string in the provided field below and click "Activate".

Computer Code:

License string:

Commuter License

A commuter license allows you to borrow / check out a network license for a certain period and use it offline, like a standalone license. It is useful for employees who frequently travel or are in locations with limited or no internet connection.

To check out a commuter license:

1. When connected to the company network that hosts the license server, enter the number of days that you need the commuter license for
2. Click **Get Commuter License**. The maximum duration is 90 days (Figure 4-4)

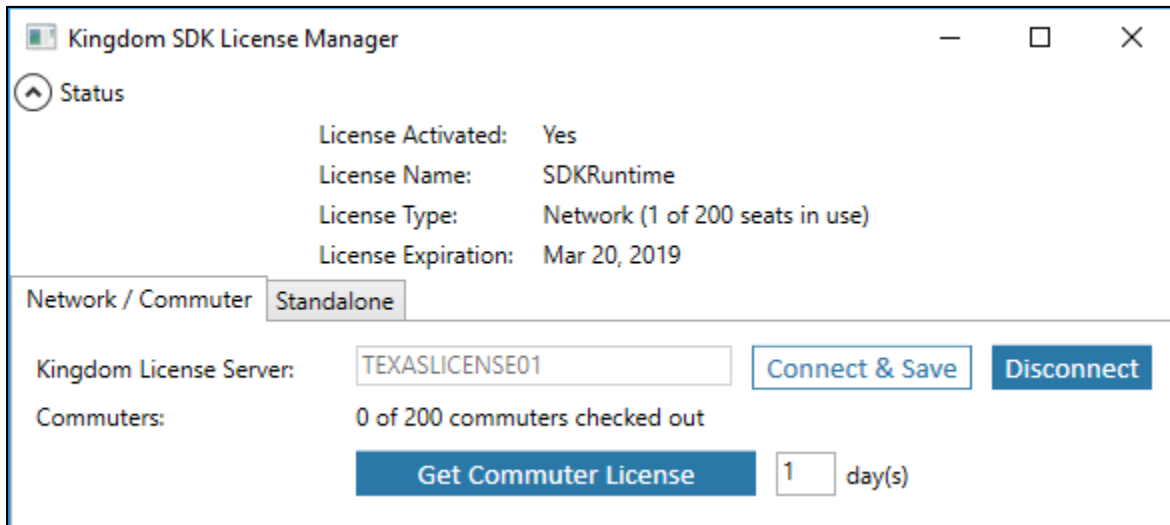


Figure 4-4: Get Commuter License

The license is automatically released after the number of days have passed. You can also release it by running the Kingdom SDK License Manager.

Connect to the license server and click **Release Commuter License**. (Figure 4-5)

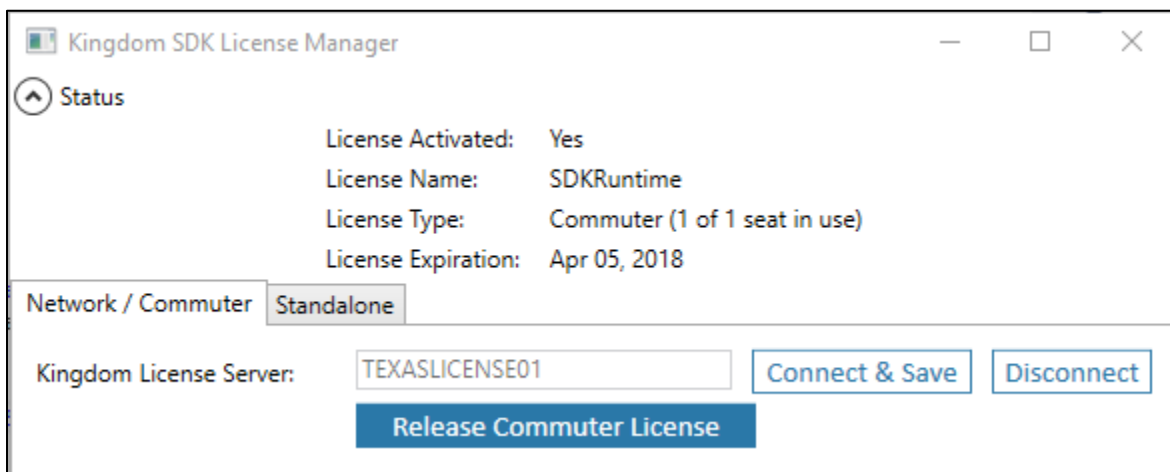


Figure 4-5: Release Commuter License

Step 2 – Set Up Project

All assemblies are available in the Assemblies folder of the Kingdom SDK package.

The following Kingdom SDK assemblies need to be referenced in your .NET project that consumes the Kingdom SDK:

- DynamicQuery.dll
- IHS.Kingdom.Licensing.dll
- IHS.Kingdom.POCO.dll
- IHS.Kingdom.WellQuery.dll
- IHS.KingdomProjectInfo.dll
- IHS.Security.Accessor.dll
- IHS.Security.Data.dll
- IHS.Security.Entities.dll
- Smt.Basics.dll
- Smt.Bulk.dll
- Smt.Data.dll
- Smt.dll
- Smt.Entities.Data.dll
- Smt.Entities.dll
- Smt.Entities.Files.dll
- Smt.IO.Brick.dll
- Smt.IO.dll
- Smt.Projection.dll
- Smt.SDK.dll
- Smt.Segy.dll
- Smt.ServiceModel.dll
- Smt.Devkit.Seismic.Interop.dll

The following Kingdom SDK assemblies need to be copied to the output folder for seismic access:

- OpenKINGDOMUtils.dll
- SeismicDevKit.dll
- Smt.Interop.Brick.dll
- Smt.Interop.Segy.dll

The following are additional third-party and Kingdom SDK assemblies that need to be copied to the output folder:

- CodeProject.Settings.dll
- DocumentFormat.OpenXml.dll
- Entity Framework v6
 - EntityFramework.dll
 - EntityFramework.SqlServer.dll
- IHS.Kingdom.SDK.EntityDbConfig.dll
- Microsoft.Practices.Prism.dll
- System.Management.Automation.dll

The following two assemblies need to be copied to the output folder if the third-party application needs to support an **Oracle-based** Kingdom project. See [Oracle](#) for details.

- Oracle.ManagedDataAccess.dll
- Oracle.ManagedDataAccess.EntityFramework.dll

The following assembly need to be copied to the output folder if the third-party application needs to support a **SQL-Based** Kingdom project:

- System.Data.SqlServerCe.dll

The following assemblies need to be copied to the output folder for licensing:

- LicenseClient.dll
- LicenseClientImplx64.dll

- LicenseClientImplx86.dll
- LicenseClientResources.dll

Step 3 - Code

Set up Project Class

Kingdom SDK is designed to work as a stand-alone library and work independently without requiring you to have Kingdom installed on the same computer. At this point, entities that are updated or inserted from a third-party application do not trigger notifications to the Kingdom application, and vice versa.

To connect to and open a Kingdom project, you first need to instantiate a Project object by setting the project path and database credentials as shown below:

```
IProject project = new Project(projectPath);  
project.DBUserName = user;  
project.SetPassword(password);  
project.LogOn(author);  
return project;
```

Parameters include:

Project Path	The file path to the Kingdom project file (.tks).
User	Database user name for your Kingdom project. If connecting to an Auto SQL Server Express (AutoSSE) project, you can leave this field blank
Password	Database password for your Kingdom project. If connecting to an AutoSSE project, you can leave this field blank
Author	The name of the author that you want to use to log on to the Kingdom project

The Project class represents a Kingdom project and contains many useful utilities as well as properties. Much of the initialization of the SDK is done within this class. The required SDK runtime license is checked out during the project object initialization.

Access Data

After a project has been initialized, you are ready to read from or write to Kingdom project data. The data context is initialized by calling GetKingdom() method, which also has an optional parameter that contains initialization settings. This topic will be covered in the next chapter.

In the following example, one well with all its boreholes is being retrieved. The intended well ID is passed as `wellId` variable and the `Get()` method returns the result depending on the defined projection.

```
using (Kingdom context = project.GetKingdom())
{
    var wellWithBoreholes = context
        .Get(
            new Well(),
            w => new { Well = w, w.Boreholes },
            w => w.Id == wellId,
            false)
        .FirstOrDefault();
}
```

More details on how the `Get()` command works and how to insert and update Kingdom project data will be covered in the next Chapter.

Oracle

The following Oracle Data Provider for .NET libraries need to be placed in the output folder:

- Oracle.ManagedDataAccess.dll
- Oracle.ManagedDataAccess.EntityFramework.dll

Deploying Your Application

As part of your application package, please include the following.

1. Licensing	Run License initialization as part of your application installation step /Utilities/lsinit.exe Configure and Activate SDK License Run /Utilities/License Activation/IHS.Kingdom.Licensing.Activate.exe and then follow the instructions on how to activate a network, standalone or commuter license.
2. Assemblies	Package all the required Kingdom SDK and third-party libraries with your application installer NOTE: There are both managed (.NET) and unmanaged libraries

Understanding Kingdom Data

Kingdom SDK API & Object Model documentation, which is included in the Kingdom SDK package, contains the definition of all available entities, entity properties, and methods for accessing the data.

Both Oracle and SQL Server database systems are supported by the Kingdom SDK.

Data Model

Supported entities are shown in the following table:

Category / Entity	Class	Read / Write	Authorized	Notes
Well Data				
Well	Well	R/W		
Borehole	Borehole	R/W		
Symbol	Symbol	R		
Borehole Note	BoreholeNoteText BoreholeNoteAttribute	R/W		
Borehole Remark	BoreholeRemark	R/W		
Well Operator	WellOperator	R/W		
Well Subset	WellSubset WellSubsetSpatial	R/W		Helpers provided in IHS.Kingdom.WellQuery .WellSubsetManager Old well subsets can be accessed via Smt.WellSubsetManager
Bottom Hole Location	BottomHoleLocationEvent	R/W		
Well Location				
Well Location	WellLocation	R/W		
Texas	WellTexas	R/W		
Offshore	WellOffshore	R/W		
Township Range	WellTownshipRange	R/W		
Well Footage	WellFootage	R/W		
NE Well & Ohio	WellNEandOhio	R/W		

Deviation Surveys				
Deviation Survey	DeviationSurvey	R/W		
Active Deviation	ActiveDeviation	R/W		
Well Plan				
Well Plan	WellPlan	R/W		
Well Plan Points	WellPlanPoint	R/W		
Well Digital Logs				
Digital Logs	LogData	R/W		Regular samples only
Log Template	LogTemplate PreferredLogTemplate	R/W	Y	*header only
Log Alias	LogAlias		Y	*
Log Name Alias	LogNameAlias		Y	*
Log Type	LogType			*
Log Curve Name	LogCurveName	R/W		
Picked Interval Name	PickedIntervalName	R/W	Y	
Picked Interval	PickedInterval	R/W	Y	
Well Raster Logs				
	RasterLogSection RasterLogFile RasterLogDepthCalibration RasterLogTrack RasterLogName	R/W		
Well Composite Logs				
Composite Logs	CompositeLogName CompositeLogAttribute CompositeLogInterval	R/W	Y	
Formation Tops				
Formation Top Pick	FormationTopPick	R/W	Y	
Formation Top Name	FormationTopName	R/W	Y	

Reserves				
Forecast	Forecast ForecastSegment ForecastTemplate	R/W	Y	
Volume	Volume VolumePolygon VolumeLayerResult VolumeLayerProperty VolumeLayer	R/W	Y	
Economics	EcoCase EcoScenario EcoProductPrice EcoAmountConstraint EcoCapex EcoCapexAmortize EcoCapexDepreciate EcoInterestConstraint EcoOpex EcoProductInterest EcoProductProduction EcoScenarioResult EcoTax EcoTaxConstraint	R/W	Y	
Production Data				
	ProducingField ProductionEntity ProductionDispositionHistory ProducingReservoir ProductionCumulativeActivityType ProductionVolumeHistory ProductionImportedCumulative ProductionCumulativeCustomized ProductionCumulativeTotal	R/W		
Time Depth Data				
	TimeDepthData	R/W		GeoSyn reads & writes to this table when opened from Kingdom

Zones				
Zone	Zone	R/W	Y	
Zone Attributes	ZoneAttributeDefinition ZoneAttributeFloat ZoneAttributeDate ZoneAttributeText	R/W	Y	
Zone Interval	ZoneInterval	R/W	Y	Updating Zone Interval needs to be done separately.
Grids				
Grid	Grid	R/W	Y	
Grid Parameters	GridParameters	R/W		
Grid Def. Params.	GridDefinitionParameters	R/W		
Contours				
	Contours	R	Y	
Cultures				
	Culture	R		
Completions				
Header	CompletionHeader	R/W		
Squeeze	CompletionSqueeze	R/W		
Perforating	CompletionPerf	R/W		
Tubing	CompletionTubing	R/W		
Tubing Equipment	TubingEquipment	R/W		
Packer	CompletionPacker	R/W		
Squeeze Type	SqueezeType	R/W		
Cut Fluid Name	CutFluidName	R/W		
Treat. Fluid Type	TreatmentFluidType	R/W		
Proppant Type	ProppantType	R/W		
Treatment Type	TreatmentType	R/W		
Treatment Status	TreatmentStatus	R/W		
Treatment	CompletionTreatment	R/W		
Packer Type	PackerType	R/W		
Tubing Eq Name	TubingEquipmentName	R/W		

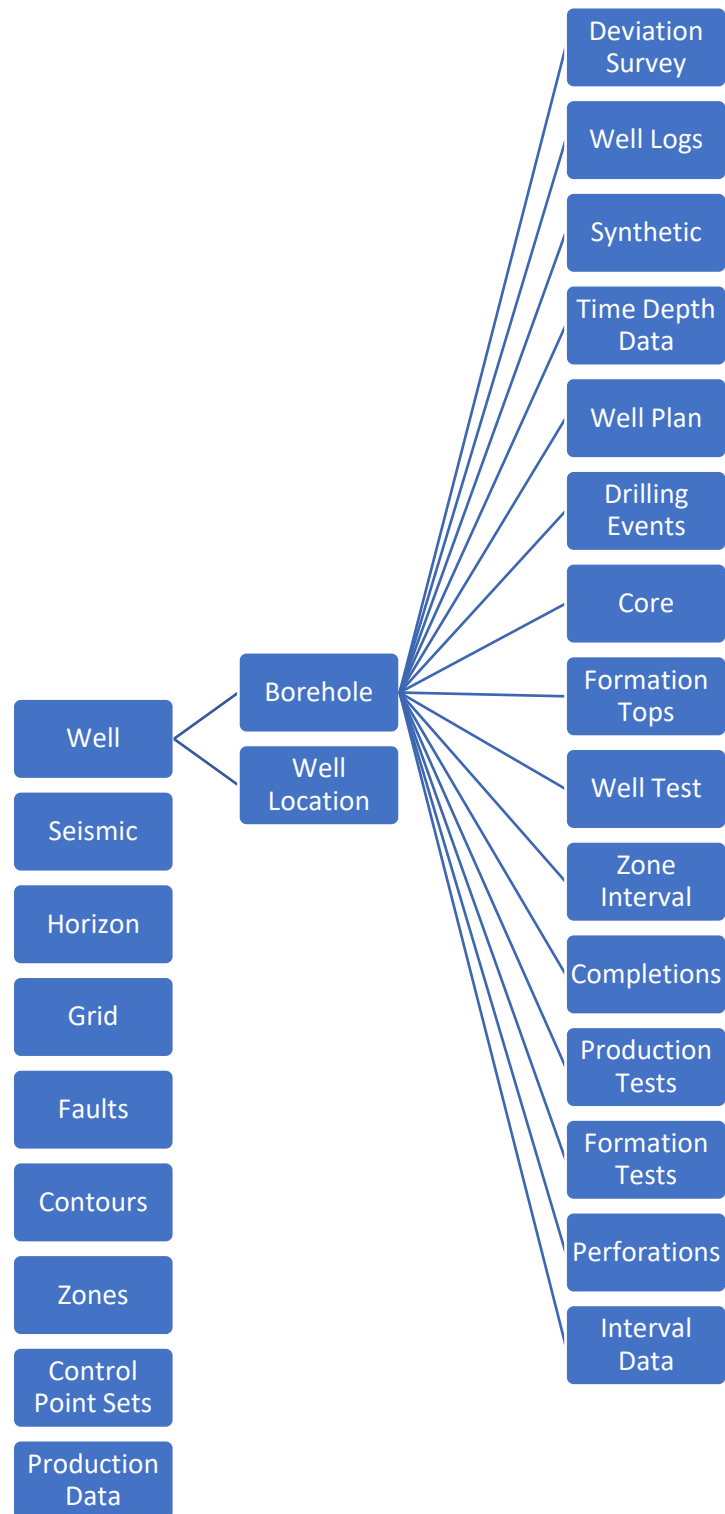
Production Tests		R/W		
Data Header	ProductionTestHeader	R/W		
Monthly Data	TestMonthlyProduction	R/W		
Initial Potential	TestInitialPotential	R/W		
Production Type	ProductionType	R/W		
Data	TestProduction	R/W		
Initial Potential Treatment	TestInitialPotentialTreatment	R/W		
Treatment	TestProductionTreatment	R/W		
Perforation	TestProductionPerforation	R/W		
Borehole Event Test Type	BoreholeEventTestType	R/W		
Formation Tests				
Formation Test	TestFormation	R/W		
Materials to Surface	TestMaterialsToSurface	R/W		
Pipe Recovery	TestPipeRecovery	R/W		
Header	FormationTestHeader	R/W		
Well Tests				
Absolute Open Flow	TestAof	R/W		
4pt AOF rate	TestAof4Point	R/W		
Bottomhole Pressure	TestBottomHolePressure	R/W		
Surface Pressure	TestSurfacePressure	R/W		
Perforations				
Perforation Status	PerforationStatus	R/W		
Perforation Type	PerforationType	R/W		
Initial Potential Perforation	TestInitialPotentialPerforation	R/W		
Synthetic				
	Synthetic	R/W	Y	GeoSyn reads & writes to this table when opened from Kingdom

Cores				
	CoreDescription CoreAnalysis CoreType CoreFluid CoreFormation CoreSampleAnalysis CoreSampleDescription	R/W		
Intervals				
	IntervalName IntervalRecord IntervalAttribute IntervalNumericValue IntervalDateTimeValue IntervalTextValue	R/W		
Faults				
Faults	Fault	R/W	Y	
Fault Cut Name	FaultCutName	R/W	Y	
Fault Cut Pick	FaultCutPick	R/W	Y	
Fault Segment	NewFaultSegment	R/W		
Fault Vertex	FaultVertex	R/W		
Fault Polygon	FaultPolygon	R/W	Y	
Drilling Events				
Casing	DrillCasing	R/W		
Liner	DrillLiner	R/W		
Mud Report	DrillMudReport	R/W		
Objective	DrillObjective	R/W		
Lost Circulation	DrillLostCirculation	R/W		
Show	DrillShow	R/W		
Log	DrillLog	R/W		
Mud Type	MudType	R/W		
Liner Type	LinerType	R/W		
Liner Grade	LinerGrade	R/W		
Hanger Type	HangerType	R/W		

Casing Type	CasingType	R/W		
Casing Grade	CasingGrade	R/W		
Casing Connection	CasingConnection	R/W		
Seismic Data				
2d Surveys	SeismicLine2D SeismicData2D	R/W		
3d Surveys	SeismicSurvey3D	R/W		
Horizons				
	Horizon	R/W	Y	
Control Points				
Control Point Set	ControlPointSet3D	R/W	Y	
CP Set Attributes	PointSetAttributes	R/W		
Misc				
Project Info	ProjectTable	R/W		
Authored Name	AuthoredName	R/W		
Namespace	Namespace	R/W		
Coordinate System	CoordinateSystem	R/W		Using BlueMarble v6.4
Datum Shift	DatumShift DatumShiftDetail	R/W		
Shared Name	SharedName	R/W		
Authored Name	AuthoredName	R/W		
GeoPhone	GeoPhone	R/W		

Data Model Categories Structure

The high-level data model categories' structure can be seen in the following diagram:



Data Access

Kingdom SDK allows developers to read, create, update, and delete objects from Kingdom data.

- **Create:** Set the CRUD parameters in the constructor of the entities with argument, or by using the default constructor which also sets the object to Create mode.

Example: `Borehole()`

- **Update:** Update operation can be set from constructor. Also, when `SetID(Id)` is used, it automatically changes the mode to Update unless the original mode was Delete. When data is read from database the mode is set to Update.

Example: `bore.SetId(1200);`

- **CreateOrUpdate:** It can only be set through the constructor.

Example: `Borehole(CRUDOption.CreateOrUpdate)`

- **Delete:** It can be set through the constructor or by calling `DeleteObject/s` API.

Example: `Borehole(CRUDOption.Delete)`

Data Context:

```
using (var kingdom = project.GetKingdom())
{
    kingdom.AddObject(bore);

    kingdom.AddObjects(new List<Source>(){source} );

    kingdom.SaveChanges();
}

project.SaveObjects(bore, zone, source)
```

- `kingdom.AddObject` – Adds one object to object state manager
- `kingdom.AddObjects` – Adds multiple objects to `ObjectStateManager`
- `kingdom.SaveChanges` – Commits changes to database/file
- `project.SaveObjects` – Allows saving different data types at the same time. By the end of this call, all the data is stored to the database/file

Read Data

By One ID

Given one ID, the following function returns one object:

```
public TType Get<TType>(int id)
```

Example:

```
using (IKingdom context = project.GetKingdom())
{
    Well well = context.Get<Well>(wellId);
}
```

By Collection of IDs

Given a collection of Ids, the following method returns IEnumerable collection of type TType:

```
public IEnumerable<TType> Get<TType>(IEnumerable<int> ids)
```

Example:

```
using (IKingdom context = project.GetKingdom())
{
    List<Well> wells = context.Get<Well>(wellIds).ToList();
}
```

Visible-only Flag

This flag determines whether to show only items which are visible in Project Explorer, or show all items. When set to true, it gets only the items which are visible. When set to false, it gets all items.

Currently, this flag needs to be set to false as the Kingdom SDK does not support integration with Kingdom yet.

```
using (IKingdom context = project.GetKingdom())
{
    List<Well> wells = context.Get<Well>(null, true).ToList();
}
```

Projection

Projection is a process of selecting data in a different shape. When querying a complete record, using projection generally results in better performance and consumes less memory than getting all object properties or list of objects. To access child/parent objects and their properties, use a navigation property/methodology as shown in the following example:

```
using (IKingdom context = project.GetKingdom())
{
    List<WellBore> wellbore = context.Get(
        new Smt.Entities.Borehole(),
        b => new WellBore
        {
            Id = b.Id,
            BoreName = b.Name,
            WellName = b.Well.Name,
            WellNumber = b.Well.WellNumber
        },
        null,
        false
    ).ToList();
}
```

Projection & Visible-only Flag

The method signature is as follows:

```
public IEnumerable<TResult> Get<TType, TResult>(
    TType entityTypeSpecifier,
    IEnumerable<int> ids,
    Expression<Func<TType, TResult>> projection,
    bool visibleOnly)
```

Predicate

Predicate is a filtering mechanism that allows developers to query entities based on conditions of the existing properties. To return all objects in the database for a query, set the predicate to null.

The predicate syntax uses .NET Lambda expression.

```
List<FormationTopName> formationTopNames;
using (IKingdom context = project.GetKingdom())
{
    formationTopNames = context.Get<FormationTopName>(null, false).ToList();
    formationTopNames = context.Get<FormationTopName>(
        t => t.OwningAuthorId == 1200, false).ToList();
}
```

Well Subsets

Subset can only be used on entities which are related to wells.

IHS.Kingdom.WellQuery.WellSubsetManager class is used to manage well subsets. Currently, the following subsets are supported by the SDK:

- New Dynamic Subsets (Kingdom 2017.0 and later)
- New Static Subsets (Kingdom 2017.0 and later)
- Old Dynamic Subsets (Kingdom 2016.1 and earlier)

Examples:

```
var subsetName = "Well Test Subset";
var wsManager = new IHS.Kingdom.WellQuery.WellSubsetManager(project);
var subsetBoreholeIds = wsManager.GetBoreholeIds(subsetName);
```

Author Types

Query can be run against different author types as shown in the following example:

```
using (IKingdom context = project.GetKingdom())
{
    List<int> formationTopNameIds = context
        .Get(
            new FormationTopName(), w => w.Id, w => w.Id > 0,
            AuthorType.VisibleAuthors,
            false)
```

```
.ToList();  
}
```

AuthorType is an Enum with following types:

- AuthorType.LoggedInAuthor
- AuthorType.VisibleAuthors
- AuthorType.AllAuthors

Author Ids

Given a collection of Author Ids, the following method returns formation top names created by those authors:

```
using (IKingdom context = project.GetKingdom())  
{  
    List<int> formationTopNameIds = context.Get(  
        new FormationTopName(),  
        w => w.Id,  
        w => w.Id > 0,  
        authorIdList,  
        false).ToList();  
}
```

Navigation Properties

Navigation properties allow for navigating from an entity to another entity that could be a single or a collection of objects. It is typically used to represent a set of children, a parent or a lookup table.

```
using (IKingdom context = project.GetKingdom())
{
    Well wellWithWellbores = context
        .Get(
            new Well(),
            w => new {Well = w, w.Boreholes},
            w => w.Id == wellId,
            false)
        .Select(x => x.Well)
        .FirstOrDefault();
}
```

```
using (IKingdom context = project.GetKingdom())
{
    DeviationSurvey deviationSurvey = context
        .Get (
            new ActiveDeviation(),
            ad => new
            {
                ActiveDeviation = ad, ad.DeviationSurvey
            },
            ad => ad.BoreholeId == boreholeId,
            false)
        .Select(ad => ad.DeviationSurvey)
        .FirstOrDefault();
}
```


Create Data

Here's an example of how to create data:

```
var project = new Project();
var bore1 = new Borehole
{
    Uwi = "ZEG34203543-4259234",
    WellId = 1000
};

var bore2 = new Borehole(CRUDOption.Create)
{
    Uwi = "ZEG34203543-4259234",
    WellId = 1000
};

using (var kingdom = project.GetKingdom())
{
    kingdom.AddObject(bore1);
    kingdom.AddObject(bore2);
    kingdom.SaveChanges();
}
```

Note:

- UWI is a natural key for Borehole
 - If UWI already exists in the database and using `CRUDOption.Create` parameter, SDK does not allow this to get saved.
-

Update Data

There are two different methods to update an object and its children. The first one is to set the natural key or primary key of the object, as shown in the following example:

```
var bore = new Borehole();
bore.SetId(1200);

// ...
// Update the borehole's properties here
// ...

using (var kingdom = project.GetKingdom())
{
    kingdom.AddObject(bore);
    kingdom.SaveChanges();
}
```

The second method is by passing the `CRUDOption.Update` parameter when instantiating a new Kingdom object, as shown below:

```
var bore = new Borehole(CRUDOption.Update) { Uwi = "ZEG34203543-4259234" };

// ...
// Update the borehole's properties here
// ...

using (var kingdom = project.GetKingdom())
{
    kingdom.AddObject(bore);
    kingdom.SaveChanges();
}
```

Create or Update Data

Alternatively, you can indicate the object is to be either inserted or updated, depending on its existence. The natural key can may be used to identify the object.

```
var bore = new Borehole(CRUDOption.CreateOrUpdate) { Uwi = "ZEG34203543-4259234" };

// ...
// Set or Update the borehole's properties here
// ...

using (var kingdom = project.GetKingdom())
{
    kingdom.AddObject(bore);
    kingdom.SaveChanges();
}
```

Delete Data

There are two different methods to delete an object.

Use a Delete Parameter

The first one is to pass the `CRUDOption.Delete` parameter when creating the object, as shown in the following example:

```
var bore = new Borehole(CRUDOption.Delete);
bore.SetId(1200);
using (var kingdom = project.GetKingdom())
{
    kingdom.AddObject(bore);
    kingdom.SaveChanges();
}
```

Call DeleteObject() Method

Call the DeleteObject() method and pass the object that you intend to delete, as shown in the following example:

```
var bore = new Borehole();
bore.SetId(1200);
using (var kingdom = project.GetKingdom())
{
    kingdom.DeleteObjects(new List<Borehole>() { bore });
    kingdom.SaveChanges();
}
```

Data Access Settings

This settings class consists of error handling and tracking configuration. It can be passed as a parameter to either of these methods:

`IProject.GetKingdom(DataAccessSettings dataAccessSettings);`

`IProject.SaveObjects(DataAccessSettings dataAccessSettings, SmtEntity[] entities);`

The class structure of data access settings is as follows:

- **DataAccessErrorHandler**
 - **ThrowException**
 - When ThrowException is set to **true** and an error occurs, an exception will be thrown. The entire transaction will be aborted and rolled back.
 - When ThrowException is set to **false** and an error occurs, no exception will be thrown and errors will be recorded under the entity objects. Entity objects without errors will be saved.
 - **DuplicateNaturalKeySelector**
- **ChangeTrackingForRead** – allows user to turn on or off tracking while getting data from database
- **DataChangedNotificationHandler** – allows users to delay sending notification when database changed. It has the following methods:
 - **DelayNotification()** – queue all the notification message without sending them out
 - **Notify()** – send all the changes in database that has been queuing and reset state to not delay state
 - **HasChangeSet()** – check if there is a pending notification

Miscellaneous Methods

Formation Tops Author Priority

Author priority aliasing uses the formation top picks of selected, prioritized authors if you do not have a pick for that top in a well. For example, assume you are Geol1 and you have a top named FT_A. You have picked this top on some wells. If you select author Geol2 as the first author in your author priority list, and Geol3 as the second author in your list, then on boreholes that do not contain your FT_A pick, Kingdom will use FT_A from Geol2. If Geol2 does not have that pick on the borehole, the application will use Geol3's pick for FT_A.

Method Signature:

```
interface IProject {  
    public IEnumerable<int> GetFormationTopAuthorPriority();  
}
```

Sample use:

```
var project = new Project(...);  
var authorIds = project.GetFormationTopAuthorPriority()
```

This method will get a list of Author IDs for the current Author's Formation Top Author Priority.

Get Formation Tops in a Strat Column

If you want to view all or a subset of your tops in stratigraphic order, a strat column can be created in Kingdom by selecting the formation tops and then ordering them from the top down, youngest to oldest.

The following method is available to retrieve tops in the selected strat column.

```
var source = new KingdomItemsSource(project, null);  
source.GetFormationTopsInStratColumn();
```

Get Formation Tops aliasing

Aliasing enables you to set preferences for using other Kingdom authors' tops on wells where you do not have the primary formation top picked. You can also specify one of your own tops as an alias for the primary formation top if desired.

The following method allows you to get a list of aliased formation tops for a particular formation top.

```
interface KingdomItemsSource {  
    public IEnumerable<int> GetAliasedFormationTopIdsFromId(int topId); }
```

Code Sample:

```
var source = new KingdomItemsSource(project, null);
var ids = source.GetAliasedFormationTopIdsFromId(1400);
```

API Changes

WellFootage (Same is applied to BoreholeFootage)

- WellFootage contains 3 footages, WellSectionLineFootage, WellCallFootage, WellSectionCornerFootage
- Only one footage is attached to WellFootage

Mapping:

Old	New
WellFootage	WellSectionLineFootage
	WellCallFootage
	WellSectionCornerFootage

Sample queries to show how the query would change:

	Old	New
1	<pre>var wellFootage = new WellFootage(); Kingdom.AddObject(wellFootage); Kingdom.SaveChanges();</pre>	<pre>var wellCallFootage = new WellCallFootage(); var wellFootage = new WellFootage(wellCallFootage); Kingdom.AddObject(wellFootage); Kingdom.SaveChanges();</pre>
2	<pre>var wellFootage = Kingdom.Get<WellFootage>(wellFootageId)</pre>	<pre>WellSectionLineFootage _LineFootage; WellSectionCornerFootage _CornerFootage; WellCallFootage _CallFootage;</pre>

		<pre> var wellFootage = Kingdom.Get<WellFootage>(wellFootageId) if (wellFootage.LineFootage != null) _LineFootage = wellFootage.LineFootage Else if (wellFootage.cornerFootage!= null) _CornerFootage = wellFootage.cornerFootage Else _CallFootage = wellFootage. CallFootage </pre>
--	--	---

Digital Log Data

- Log "Name" and "Abbreviation" in LogData are replaced with Navigation Property LogCurveName
- NameId in LogData is replaced with LogCurveNameId

Mapping:

Old	New
LogData.Name	LogData.LogCurveName.Name
LogData.Abbreviation	LogData.LogCurveName.Abbreviation
LogData.NameId	LogData.LogCurveNameId

Sample queries to show how the query would change:

Important to note if LogCurveName is called in the Projection of a query then it will be available to access from LogData using LogData.LogCurveName. If it is not a part of projection in the query then LogData.LogCurveName will be null.

In Predicate SDK can handle LogData.LogCurveName.Name but unless it is not asked in Projection user code will have LogCurveName under LogData as NULL.

	Old	New
1	<pre> var logData = kingdom.Get<LogData>(logDataId); var logName = logData.Name; </pre>	<pre> Method 1: var logDataWayOne = kingdom.Get(new LogData(), logData => new { logData, logData.LogCurveName}, x => x.Id == logDataId, false).Select(x=>x.logData).FirstOrDefault(); var logName = logDataWayOne.LogCurveName.Name; </pre>

		Method 2: //We recommend caching of LogCurveName (Name Id) information when using this method <pre>var logDataWayTwo = kingdom.Get<LogData>(logDataId); var logName = kingdom.Get<LogCurveName>(logDataWayTwo.LogCurveNameId).Name;</pre>
2	<pre>var logData = kingdom.Get<LogData>(x => x.Name == "GR" && x.BoreholeId = boreholeId, false).FirstOrDefault();</pre>	<pre>var logData = kingdom.Get<LogData>(x => x.LogCurveName.Name == "GR" && x.BoreholeId = boreholeId, false).FirstOrDefault();</pre> Note: LogData in this query returns the result of the Log with matches the LogName and belongs to given Borehole. However, when we access LogCurveName under LogData using LogData.LogCurveName will return NULL as there is no projection in this query.

Programming Guidelines and Best Practices

Instantiate a single Project object	A single instance of the project should be used, If possible, during the course of the application session. A singleton or service pattern can be used to keep the project object alive. At the end of the lifespan, the project object should be disposed of to clear both managed and unmanaged resources.
Chunk items to 300 records on saving	To optimize performance, we recommend saving maximum of 300 records at a time.
Associations help save all related objects	Saving a Borehole object saves its related zone object(s), zoneAttribDef object(s), zoneInterval object(s), and zoneAttributeFloat object(s). Refer: FuncSavingOneSavesAll (get diagram)
Dispose project object after use	The project object should be disposed when it is no longer needed due to some unmanaged third-party library usage by the SDK. For example, you might want to do this when your application closes its Kingdom connector module.
Do not use single context for multiple operations	Refer: FuncSaveChangesInAContext
Limit operations inside the context	Limit operations inside the context to context-related operations, i.e. add/delete/update/edit and the save changes action. Mapping objects to and

	from the Kingdom objects and other non-related operations should be done outside of the context.
Unique key/ID required for 2 objects in the same context	SDK does not allow two objects of one data type to have same natural key or Id under the same context. Refer: FuncSameNaturalKeyOrIdUnderOneContext
Updating/deleting by ID/Key	Use the Id or Natural key to identify the database record (Id takes the precedence) For Update set Id/Natural key and the values that need to be updated For Delete set Id/Natural key Refer: PerfUpdateDeleteTests
Deleting parent objects	Deleting a parent object deletes all children. When a zone is deleted through SDK, SDK will clean ZoneAttribDef, ZoneInterval, ZoneAttributeFloat Refer: FuncDeleteParentDeletesChildren (Get Diagram)
Use Projection for a single entity	When querying a complete record, using projection has a better performance than getting the whole object. Refer: PerfProjection, FuncGetOperations
Use Projections carefully	Too many navigation properties cannot be handled by Oracle. Refer: FuncMoreNavigationItemsInProjection
Use Contains carefully	Using contains with strings is a big performance hit Refer: PerfUsingContainsWithStrings

Appendix

Appendix A – Exceptions

SQL Server Express Exceptions (SseException)

Exception Value or Message	Notes
InvalidDatabase	Check if the Kingdom project database is valid
ServerNotInstalled	The SMTKINGDOM instance of SQL Server Express is not installed on this machine. Please install and try again or move the project to network.
SqlServerIsNotRunning	SQL Server is not running
MoreConnections	Some users or applications are still connected to the project. The project will not be portable to copy or move until all connected applications are closed.
UnableToUncompressProjectDatabaseFolder	The project database directory must be uncompressed before it can be used by SQL Server.
	This project is still attached to SQL Server Express. Either the project is still in use or the project did not close properly. Close any open projects, verify there are no Direct Connect jobs in progress, and try again. If this does not resolve the issue, re-open and then close the project using Kingdom version 8.6, 8.7, 8.7.1, or 8.8, and then try the upgrade again.
	This project is not an Auto SSE project

Additional Exceptions

Exception Value or Message	Notes
InvalidOperationException	SDK runtime license is not present when creating a Kingdom context. Please make sure that your license is setup correctly or please contact Kingdom
UnauthorizedAccessException	SDK runtime license is not present during initialization of a Project object
DuplicatePrimaryKeyException	
ParseException	Within the DynamicQuery class, whenever a query is corrupted or not supported
SaveToDatabaseFailedException	During BulkSave, failed saving data to database, and transaction is rolled back. It could be due to various reasons, such as: database permission, role-based security permission and missing required field.
KingdomDataException	
DuplicatePrimaryKeyException	A duplicate primary key found during update or insert
MissingParentException	During preprocessing in Bulk Save, the parent of an existing object is missing